

SYSTEM DESIGN INTERVIEW

Design a Calendar Service

Google/Outlook-style scheduling — events, recurrences, sharing, free-busy

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Data Model & Schema
- 06 · Recurrence: RFC 5545 RRULE
- 07 · Recurrence Expansion Algorithm
- 08 · Time-Zone Correctness
- 09 · Sharing & ACL
- 10 · Free-Busy & Meeting Picker

- [11](#) · Notifications & Reminders
- [12](#) · iCalendar & CalDAV Interop
- [13](#) · Storage & Sharding Strategies
- [14](#) · Failure Modes & Trade-offs
- [15](#) · Interview Playbook

01

Problem Statement

Requirements and scope

Design a calendar service like **Google Calendar** or **Microsoft Outlook**. Users create events with optional recurrence, share calendars, schedule meetings against attendees' free-busy, and receive reminders. The service must handle complex recurrence rules, time zones, and read-heavy access patterns at global scale.

Clarifying Questions

Question	Answer
Active users?	~500M monthly; ~100M concurrent at peak
Events/user/week?	~10 (mix of one-off and recurring masters)
Recurrence?	Yes — RFC 5545 RRULE (FREQ, INTERVAL, BYDAY, COUNT, UNTIL)
Sharing?	Per-calendar ACL + per-event override; delegate calendars
Free-busy?	Yes — meeting picker queries union of busy intervals
Notifications?	Per-event reminders (5 min, 1 day, custom); email + push
Interop?	iCalendar (.ics) export/import; CalDAV for IMAP/Mac/Thunderbird
Time zones?	Full IANA TZ DB; correct around DST transitions

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Create / edit / delete event	Title, time range, location, attendees, RRULE, reminders
Recurrence	RFC 5545 RRULE; per-instance exceptions (move/cancel one occurrence)
List view	Day / week / month query: events overlapping window [t1, t2]
Sharing	Per-calendar ACL roles: free-busy / see-all / edit / owner
Meeting scheduling	Free-busy union over N attendees → pick available slot
Notifications	Reminders 5 min / 1 day / custom; email + push; idempotent
Interop	iCalendar export/import; CalDAV PROPFIND/REPORT

Non-Functional Requirements

Requirement	Target
Availability	99.95% — calendars are mission-critical for enterprise
Latency	<150 ms event lookup; <500 ms free-busy over 10 attendees
Consistency	Strong within a calendar; eventual across free-busy aggregation
Read:Write ratio	~50:1 (people glance at calendars constantly)
Throughput	~500K event lookups/sec at peak; ~10K writes/sec
Storage	~1B active events; ~5 TB hot + attachments in object store
Time-zone correctness	All IANA TZs; DST-safe recurrence expansion

03

Capacity Estimation

Math for scale

Traffic Estimation

- Active users: **500M**
- Avg events created: **10/user/week** → ~700M events/week → ~100M/day
- Total active events (window of ~10 weeks): **~1B events**
- Write throughput: $100M / 86,400 = \sim 1.2K \text{ writes/sec}$ avg; **~10K/sec at peak**
- Read throughput (50:1 ratio): **~500K event lookups/sec at peak**
- Free-busy queries (meeting picker): ~5K/sec, each fanning out to ~10 calendars

Storage Estimation

- Per event row: ~2 KB (title, body, attendees JSON, RRULE, ACL refs)
- 1B events × 2 KB ≈ **2 TB** primary; replicas + indexes ≈ **5 TB**
- Exceptions table (modified single instances of recurrences): ~10% of recurring masters
- Attachments: object store (S3/GCS); avg 200 KB × 5% events ≈ **10 TB attachments**

Cache Estimation

- Hot working set: events overlapping the next 4 weeks for active users
- ~100M concurrent users × ~30 events visible ≈ **3B event references**
- Compressed expanded-occurrence cache: ~150 GB Redis cluster (TTL = 1 h)
- Free-busy intervals are far smaller (just busy spans): ~20 GB Redis

KEY

Key Numbers

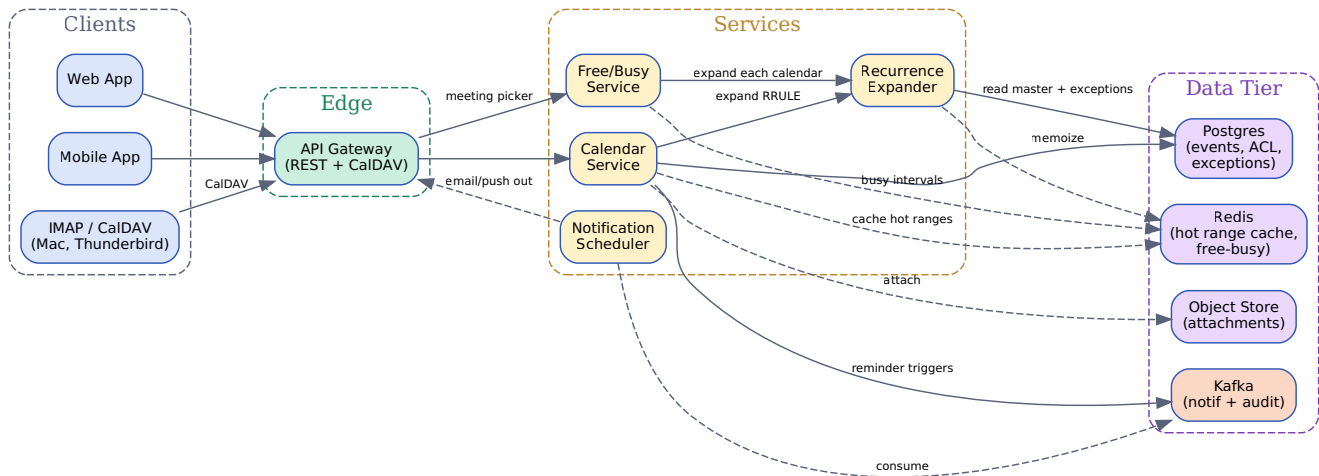
500M users · ~10 events/user/week · ~1B active events · **500K reads/sec peak** · 10K writes/sec peak
· ~5 TB hot + 10 TB attachments · ~150 GB occurrence cache.

04

High-Level Architecture

System overview

The system splits into **API Gateway**, four core services (**Calendar**, **Recurrence Expander**, **Free/Busy**, **Notification Scheduler**), and a data tier of Postgres (events), Redis (hot ranges), Kafka (notification triggers and audit), and an object store (attachments). Reads dominate; the Recurrence Expander is the hottest internal service.



Clients hit API; lookups expand recurrences lazily. Free-busy aggregates over attendees. Notifications fan out via Kafka.

Architecture Highlights

- **API Gateway:** REST for web/mobile; CalDAV (PROPFIND/REPORT) for IMAP-style clients
- **Calendar Service:** CRUD on events; enforces ACL; emits reminder triggers
- **Recurrence Expander:** takes RRULE master + window $[t1, t2]$ → list of occurrences
- **Free/Busy Service:** fans out to N attendee calendars; returns interval union
- **Notification Scheduler:** consumes Kafka; fires email/push at the right time
- **Postgres:** partitioned by user_id; recurrence master + exceptions overlay
- **Redis:** caches expanded ranges and busy intervals (1 h TTL)
- **Kafka:** reminder trigger stream and audit log; dedup by (event_id, occurrence_ts)
- **Object store:** S3/GCS for attachments referenced by event row

05

Data Model & Schema

events, exceptions, ACL

Core Tables

```
-- Calendar (one user may own many: primary, work, family)
CREATE TABLE calendars (
  calendar_id  UUID PRIMARY KEY,
  owner_user_id BIGINT NOT NULL,
  name         TEXT NOT NULL,
  default_tz   TEXT NOT NULL,          -- IANA, e.g. 'America/Los_Angeles'
  is_secret    BOOLEAN DEFAULT FALSE,  -- delegate-only calendars
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

-- Recurrence master OR a single one-off event
CREATE TABLE events (
  event_id      UUID PRIMARY KEY,
  calendar_id   UUID NOT NULL REFERENCES calendars,
  user_id       BIGINT NOT NULL,        -- partition key
  title         TEXT NOT NULL,
  body          TEXT,
  location      TEXT,
  start_utc     TIMESTAMPTZ NOT NULL,
  end_utc       TIMESTAMPTZ NOT NULL,
  origin_tz     TEXT NOT NULL,          -- IANA TZ for RRULE expansion
  rrule         TEXT,                  -- RFC 5545 RRULE; NULL = one-off
  rdate         TEXT[],                -- extra explicit dates
  exdate        TEXT[],                -- excluded occurrences
  attendees     JSONB,                 -- [{user_id, response}]
  reminders     JSONB,                 -- [{minutes_before, channel}]
  attachments   JSONB,                 -- [{key, name, size}]
  updated_at    TIMESTAMPTZ DEFAULT NOW(),
  INDEX (user_id, start_utc),
  INDEX (calendar_id, start_utc)
) PARTITION BY HASH (user_id);

-- Per-instance exception (modified or cancelled occurrence)
CREATE TABLE event_exceptions (
  event_id      UUID NOT NULL REFERENCES events,
  occurrence_utc TIMESTAMPTZ NOT NULL,  -- original start of the occurrence
  is_cancelled  BOOLEAN DEFAULT FALSE,
  override      JSONB,                  -- new {start, end, title, ...}
  PRIMARY KEY (event_id, occurrence_utc)
);
```

```
-- Per-calendar ACL with per-event override
CREATE TABLE calendar_acl (
  calendar_id  UUID NOT NULL REFERENCES calendars,
  grantee_id   BIGINT NOT NULL,
  role         TEXT NOT NULL,           -- free_busy | see_all | edit | owner
  PRIMARY KEY (calendar_id, grantee_id)
);

CREATE TABLE event_acl_override (
  event_id     UUID NOT NULL REFERENCES events,
  grantee_id   BIGINT NOT NULL,
  role         TEXT NOT NULL,
  PRIMARY KEY (event_id, grantee_id)
);
```

Why Partition by user_id

- **Locality:** day/week/month queries always filter by user → single shard
- **Even spread:** hash partition avoids hotspots from celebrity calendars
- **Free-busy fan-out:** N attendees → N parallel single-shard reads (cheap)
- **Re-sharding:** add partitions; Postgres native HASH partitioning handles it

INSIGHT

Master + Exceptions Pattern

A weekly meeting for 5 years is **one row** in events with an RRULE — not 260 rows. If one occurrence moves an hour, we add one row in event_exceptions. Storage stays bounded; reads expand the master and overlay exceptions on demand.

06

Recurrence: RFC 5545 RRULE

FREQ, INTERVAL, BYDAY, COUNT, UNTIL

What an RRULE Looks Like

```
# Every Monday and Wednesday at 10:00 in America/Los_Angeles, until end of 2026
DTSTART;TZID=America/Los_Angeles:20260105T100000
RRULE:FREQ=WEEKLY;INTERVAL=1;BYDAY=MO,WE;UNTIL=20261231T235959Z

# Last Friday of every month, 12 occurrences
RRULE:FREQ=MONTHLY;BYDAY=-1FR;COUNT=12

# Every 2 years on Feb 29 (leap years only) – RFC 5545 silently drops invalid dates
RRULE:FREQ=YEARLY;INTERVAL=2;BYMONTH=2;BYMONTHDAY=29
```

RRULE Parts We Support

Part	Meaning	Example
FREQ	Frequency: SECONDLY..YEARLY	WEEKLY
INTERVAL	Step between intervals	INTERVAL=2 → every other week
BYDAY	Day-of-week filter	BYDAY=MO,WE,FR; -1FR = last Friday
BYMONTHDAY	Day-of-month filter	BYMONTHDAY=15
BYMONTH	Month filter	BYMONTH=2 → February only
BYSETPOS	Pick the n-th match in interval	-1 → last; 1 → first
COUNT	Stop after N occurrences	COUNT=12
UNTIL	Stop after a specific UTC instant	UNTIL=20261231T235959Z

Lazy vs Eager Expansion — the Right Default

Strategy	Pros	Cons
Eager (materialise every occurrence)	Trivial range scan; no expansion at read	1 row × 5 yrs weekly = 260 rows; storage blows up; updating an RRULE means rewriting hundreds of rows; century-spanning rules unbounded

Strategy	Pros	Cons
Lazy (expand on read for window [t1,t2])	1 row per master; cheap edits; bounded by query window; matches calendar UX	CPU cost on read; cache the result; needs a robust RRULE iterator library
Hybrid (lazy + materialise popular ranges)	Lazy default; pre-materialise next-30-days for frequent rules into Redis	Two-tier cache to keep coherent on edit

TIP

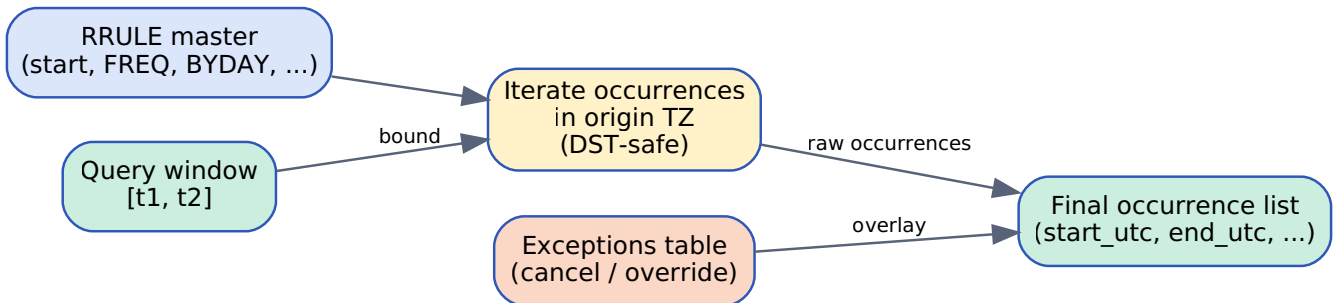
Default = Lazy + Cache

Store the recurrence as **one master row**; expand on read for the requested window; memoize results in Redis with a 1 h TTL keyed on (event_id, t1, t2, version). Pre-materialise only the next 30 days for hot calendars (heuristic: viewed > 100×/day).

07

Recurrence Expansion Algorithm

DST-safe iteration



Expansion is master-expand → exception-overlay. Cancellations remove; modifications replace.

Pseudo-code

```

from zoneinfo import ZoneInfo
from dateutil.rrule import rrulestr

def expand(master, t1_utc, t2_utc, exceptions):
    """Expand an RRULE master in window [t1_utc, t2_utc].

    Iterate in the originating TZ so DST shifts don't drift the wall-clock
    time of weekly meetings (10am stays 10am even after a DST flip).
    """
    tz = ZoneInfo(master.origin_tz)
    dtstart_local = master.start_utc.astimezone(tz)

    rule = rrulestr(master.rrule, dtstart=dtstart_local)
    duration = master.end_utc - master.start_utc

    # rrule yields LOCAL datetimes (DST-aware). Convert each back to UTC.
    occurrences = []
    for local_start in rule.between(
        t1_utc.astimezone(tz), t2_utc.astimezone(tz), inc=True):
        start_utc = local_start.astimezone(ZoneInfo('UTC'))
        end_utc = start_utc + duration
        occurrences.append({
            'event_id': master.event_id,
            'occurrence_utc': start_utc,
            'start_utc': start_utc, 'end_utc': end_utc,
            'title': master.title,
        })

    # Overlay exceptions: cancel removes; override replaces in place.
  
```

```

by_key = {o['occurrence_utc']: o for o in occurrences}
for ex in exceptions:
    if ex.is_cancelled:
        by_key.pop(ex.occurrence_utc, None)
    else:
        slot = by_key.get(ex.occurrence_utc)
        if slot:
            slot.update(ex.override)

return sorted(by_key.values(), key=lambda o: o['start_utc'])

```

Why Iterate in the Originating TZ

- **DST drift:** a 10am Los Angeles meeting set in February is 18:00 UTC; after the March DST flip the same wall-clock 10am is 17:00 UTC. Iterating in UTC would silently shift the meeting an hour twice a year.
- **RFC 5545 mandates** RRULE recurrence be computed against DTSTART's local TZ.
- **Render in viewer's TZ:** after expansion, present in the viewing user's TZ (which may differ from the originating TZ).
- **All-day events** are TZ-floating (a date, not an instant) — store as DATE not TIMESTAMPTZ; expand without TZ conversion.

08

Time-Zone Correctness

UTC + IANA TZ + DST-safe

Storage Convention

- **Wire format:** all timestamps stored as TIMESTAMPTZ in UTC
- **Originating TZ:** stored alongside as IANA string (e.g. 'America/Los_Angeles') — never as a fixed offset like -08:00
- **RRULE expansion** uses the originating TZ to handle DST
- **Render** in the viewing user's preference (their TZ, locale, 12/24 h)

Why Not a Fixed Offset

- **Offsets change:** Pacific is UTC-8 in winter, UTC-7 in summer — the IANA name encodes the rule, the offset is a snapshot
- **Governments change rules:** the IANA TZ DB is updated several times a year; you ship updates to all servers
- **Out-of-date TZ DB** = silently wrong meeting times for users in regions that changed DST policy (a recurring outage class)

Worked Example: DST Spring-Forward

Date	Local 10:00 AM (LA)	UTC	Notes
Mar 7 (PST = UTC-8)	10:00 AM	18:00 UTC	Pre-DST
Mar 14 (PDT = UTC-7)	10:00 AM	17:00 UTC	DST forward
Mar 21 (PDT = UTC-7)	10:00 AM	17:00 UTC	Stable

WARNING

DST Edge Cases

Spring forward: 2:30 AM doesn't exist on the DST day — clients must reject or re-prompt. **Fall back:** 1:30 AM happens twice — ambiguous; we pick the first occurrence (pre-transition) by convention. Same applies to **government policy changes:** when a country abolishes DST, recurring events from before the change shift unless we re-anchor.

09

Sharing & ACL

Per-calendar + per-event override

ACL Roles

Role	Capabilities
free_busy	See only busy/free intervals; no titles, no attendees
see_all	Read all event details (title, location, body) — no edits
edit	Create / modify / delete events on the calendar
owner	All edit rights + manage ACL + delete calendar

Two-Layer Resolution

1. Resolve **calendar-level role** for the requesting user (or 'none')
2. If the event has an entry in `event_acl_override` for the user, that role wins
3. If both are 'none', the event is invisible (404, not 403, to avoid leaking existence)
4. **free-busy** + override = 'see_all' is the typical 'I let my boss see this private event'

Secret / Delegate Calendars

- **Secret calendar:** `is_secret=true`; never appears in directory search; access only via direct ACL grant
- **Delegate access:** assistants schedule on behalf of executives — stored as a calendar-level `edit` grant with `delegate=true` flag
- **Audit log:** every ACL change emits a Kafka audit event; retained 2 yr
- **Acceptance flow:** share invite is a notification; recipient must accept before grant becomes active (prevents harassment via auto-share)

INSIGHT

ACL Granularity Trade-off

Per-event ACL is powerful but expensive: every event read needs an ACL lookup. Cache the calendar-level role aggressively; only consult per-event override on a miss flag (`has_event_overrides` bit on the event row).

10

Free-Busy & Meeting Picker

Union of busy intervals across N attendees

What the Picker Needs

Given a set of attendee user_ids and a target window (e.g. 'next Monday 9–5, 1 hour duration'), return the candidate slots when **everyone is free**. Output should be the inverse of the union of all attendees' busy intervals, filtered to slots $\geq$ duration.

Algorithm

```
def free_slots(attendees, t1, t2, duration_min):
    """Find common free slots  $\geq$  duration_min within [t1, t2]."""
    # 1. Fan out: pull busy intervals from each attendee's calendar
    busy_per_user = parallel_map(
        lambda u: list_busy_intervals(u, t1, t2), # uses ACL=free_busy
        attendees,
    )

    # 2. Flatten and merge overlapping intervals (sweep line)
    intervals = sorted([iv for ivs in busy_per_user for iv in ivs])
    merged = []
    for s, e in intervals:
        if merged and s <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))

    # 3. Invert to free intervals within [t1, t2]
    free, cursor = [], t1
    for s, e in merged:
        if s > cursor: free.append((cursor, s))
        cursor = max(cursor, e)
    if cursor < t2: free.append((cursor, t2))

    # 4. Filter to slots big enough for the meeting
    return [(s, e) for (s, e) in free if (e - s).total_seconds() >= duration_min*60]
```

Optimisations

- **Cache busy intervals** per (user_id, day) in Redis with 5 min TTL — viewing-window picker reads are bursty

- **Bitset trick:** bucket the day into 5 min slots → 288 bits → 36 bytes; AND across attendees finds free slots in $O(N)$ per day
- **ACL-aware:** Free/Busy service strips titles — even `see_all` users see only busy spans here, by design
- **Eventual consistency OK:** a 5 min stale view is fine for the picker; we re-validate on the actual booking write

11

Notifications & Reminders

Kafka fan-out, idempotent dedup

Reminder Spec

- Per-event reminder list: [{minutes_before: 5, channel: 'push'}, {minutes_before: 1440, channel: 'email'}]
- **Default:** 10 min push (configurable per calendar)
- **Custom:** user can add up to 5 reminders per event
- **Snooze:** client-side; backend sends once per occurrence

Pipeline

1. On event create/edit, Calendar Service emits a reminder_trigger Kafka record per (event_id, occurrence_utc, channel) for the next 30 days
2. Notification Scheduler is a delayed-queue consumer (e.g. Kafka + RocksDB timer) that holds messages until fire_at = occurrence_utc - minutes_before
3. At fire time, dispatcher calls Email/Push services with idempotency key (event_id, occurrence_utc, channel, reminder_id)
4. Email/Push services dedup on the idempotency key (24 h window) so retries don't double-send
5. Recurring events: each occurrence within the 30 day horizon gets its own trigger; a daily refresh job extends the horizon

Idempotency & Dedup

```
# Idempotency key for a reminder dispatch
key = sha1(f"{event_id}:{occurrence_utc.isoformat()}:{channel}:{reminder_id}")

# Dispatcher checks Redis SET-IF-NOT-EXISTS (NX)
if redis.set(f'rem:{key}', 1, nx=True, ex=86400):
    send_email_or_push(...)
else:
    # already sent within the last 24 h – drop silently
    metrics.incr('reminder.deduped')
```

TIP

Why a Horizon Window

Materialising every reminder for a 5-year weekly meeting is wasteful. We schedule only the next 30 days and run a nightly job that walks recurrence masters with `last_horizon_extended < today` and emits the next batch. Edits invalidate the horizon and re-emit.

12

iCalendar & CalDAV Interop

.ics export/import; PROPFIND/REPORT

iCalendar (.ics)

- **RFC 5545 file format:** text-based VCALENDAR / VEVENT records
- **Export:** any event or calendar can be downloaded as .ics — same RRULE we already store, plus DTSTART/DTEND/SUMMARY/LOCATION/ATTENDEE
- **Import:** parse with a vendored library (icalendar4j / ical.py); validate; reject events with unknown extensions; map to our schema
- **Email invitations** are .ics attachments with METHOD=REQUEST

Sample VEVENT

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//Example//Calendar 1.0//EN
BEGIN:VEVENT
UID:550e8400-e29b-41d4-a716-446655440000@example.com
DTSTAMP:20260506T120000Z
DTSTART;TZID=America/Los_Angeles:20260511T100000
DTEND;TZID=America/Los_Angeles:20260511T103000
RRULE:FREQ=WEEKLY;BYDAY=MO,WE;UNTIL=20261231T235959Z
SUMMARY:Standup
LOCATION:Zoom
ATTENDEE;CN=Alice;PARTSTAT=ACCEPTED:mailto:alice@example.com
ATTENDEE;CN=Bob;PARTSTAT=NEEDS-ACTION:mailto:bob@example.com
END:VEVENT
END:VCALENDAR
```

CalDAV (RFC 4791)

- **HTTP-based** protocol on top of WebDAV: PROPFIND, REPORT, PUT, DELETE
- **Discovery:** client does PROPFIND on principal URL → finds calendar collection URL
- **Time-range REPORT:** client asks 'give me events in [t1, t2]' → we expand recurrences and return matching VEVENTs
- **ETag** on each event for optimistic concurrency on writes
- **Sync token (RFC 6578):** incremental sync — client says 'give me everything that changed since token X'

TIP

Apple Mail / Thunderbird Just Work

Implementing CalDAV correctly means every IMAP-style desktop client works out of the box. The hard part is faithful **VTIMEZONE** handling for cross-vendor recurrence.

13

Storage & Sharding Strategies

Comparison and rationale

Comparison of Storage Approaches

Strategy	Pros	Cons
Single SQL, no shard	Simple; full ACID; joins for ACL	Doesn't scale past one box; 5 TB hot is too much for one instance
Postgres partition by user_id (chosen)	Range queries hit one shard; ACL joins local; native HASH partitioning	Cross-user free-busy = N parallel reads (fine, bounded N)
NoSQL key-value (DynamoDB)	Trivially scales; cheap reads	No joins (denormalize ACL); harder for time-range scans
Time-series DB (Cassandra)	Great for append-only event log	Updates and per-row ACL are awkward
Wide-column with partition by (user_id, month)	Time-bucketed scans; old months age out cheaply	Recurring events crossing months duplicate metadata

Hot vs Cold

- **Hot:** events with `start_utc` in the next 90 days OR last 14 days — Postgres
- **Cold:** older events archived to a read-only Postgres replica or S3+Parquet (occasional access, no edits)
- **Recurrence masters never go cold** — they generate occurrences forever
- **Attachments** always in object store; events store only S3 keys + size + mime

Read Path Optimisations

- **Range cache:** Redis key = `(user_id, week_start) → [event_ids]`; TTL 1 h
- **Expansion cache:** per-master keyed on `(event_id, week_start, version)`; version invalidates on edit
- **Bulk RRULE expand:** for a week view, expand all of a user's recurring masters in one batch; helps L1 cache locality
- **Read replicas** for free-busy fan-out (eventual consistency is fine)

14

Failure Modes & Trade-offs

What can go wrong

Failure	Impact	Detection	Mitigation
TZ DB out of date	Recurring events shift silently after a country's DST policy change	Drift alarms; user reports	Auto-deploy IANA tzdata 2x / month; canary one region first; emit warning when DTSTART falls in a DST gap
Recurrence rule with no UNTIL/ COUNT	Expansion can iterate forever; CPU exhaustion	Per-request occurrence cap (e.g. 1000)	Validate on write: reject > century span without COUNT/UNTIL; cap iterations
Leap second / leap year edge cases	Feb 29 yearly rules; 23:59:60 UTC second	Unit tests; date math review	RFC 5545 specifies dropping invalid dates (Feb 29 in non-leap years); store UTC, ignore leap seconds for scheduling
Postgres partition hot-spot	One celebrity calendar overwhelms a partition	Per-partition QPS metrics	Move calendar to dedicated partition; cache aggressively at edge
Notification dispatcher lag	Reminders fire late	End-to-end fire latency SLO	Auto-scale on Kafka lag; tier critical reminders (< 15 min) on a separate fast lane
Free-busy stale	Picker shows free, but slot just got booked	Re-validate on write	Optimistic write: if conflict, return 409 + alternate slots
Object store outage	Attachments inaccessible	Health check	Event itself remains usable; UI shows 'attachment unavailable'

Trade-offs

Decision	Choice	Trade-off
Recurrence storage	Lazy expand from master + exceptions	Storage tiny; CPU cost on read (mitigated by Redis cache). Eager would blow up storage and edit cost.
ACL granularity	Per-calendar default + per-event override	Two lookups per event in worst case. Per-event-only would be fully expressive but every read paid the override cost.
Free-busy consistency	Eventual (5 min stale)	Picker is fast and cacheable; rare double-book caught by optimistic write.

Decision	Choice	Trade-off
Time storage	UTC + IANA TZ name	Two columns instead of one offset; correct across DST + policy changes.
Notification horizon	30 days, refreshed nightly	Bounded queue size; recurring events past 30 days need the refresh job to be healthy.
Partition key	user_id (hash)	Ranges hit one shard; cross-user free-busy is N parallel reads (bounded by attendees).

WARNING

Graceful Degradation

Calendar reads are the critical path. If Recurrence Expander is down, serve non-recurring events directly from Postgres so users still see today's agenda — better than a blank page. If Notifications are down, log and drop; users complain about late reminders less than a 500-page calendar.

15

Interview Playbook

How to present this

Calendar is a deceptively deep system: most candidates breeze through CRUD and stumble on recurrence + time zones. Lead with the data model, then RRULE expansion, then free-busy. Save sharing/notifications for the deep dive.

45-Minute Outline

1. **Intro (2 min):** 500M users, ~1B events, 500K reads/sec, 10K writes/sec, recurrences + TZs
2. **Capacity (4 min):** ~5 TB hot + 10 TB attachments; 50:1 read:write
3. **High-level arch (4 min):** API → Calendar / Expander / Free-Busy / Notifier → Postgres / Redis / Kafka / S3
4. **Schema (5 min):** events (with rrule), exceptions, calendar_acl; partition by user_id
5. **Recurrence (10 min):** RFC 5545; lazy vs eager debate; DST-safe iteration in originating TZ
6. **Free-busy (6 min):** fan-out across attendees; sweep-line interval merge; Redis cache
7. **Sharing (4 min):** 4 ACL roles; per-event override; secret/delegate calendars
8. **Notifications (4 min):** Kafka + delayed dispatcher; idempotency key; 30-day horizon
9. **Failures (3 min):** TZ DB stale, runaway recurrence, partition hotspot
10. **Wrap (3 min):** trade-offs and what you'd build next (search, AI scheduling)

Key Talking Points

- “Lazy expansion is the default” — defend with storage cost + edit cost arguments
- “Iterate in the originating TZ, store UTC” — DST-safe by construction
- “UTC + IANA name, never a fixed offset” — survives policy changes
- “Master row + exceptions overlay” — bounded storage for 5-year weekly meetings
- “Per-calendar ACL + per-event override” — covers 99% of cases at low read cost
- “Free-busy is the inverse of the union of busy intervals” — sweep-line, $O(N \log N)$
- “30-day reminder horizon, refreshed nightly” — bounds Kafka queue size
- “Idempotency key (event_id, occurrence_utc, channel)” — exactly-once-ish dispatch

Common Follow-ups & Answers

- **Q: Why not materialise recurrences?** A: A 5-year weekly meeting is 260 rows instead of 1; editing the RRULE rewrites all of them; storage and write amplification are both bad. We materialise only the 30-day notification horizon.

- **Q: How do you handle DST?** A: Store UTC + IANA name; expand RRULE in the originating TZ so 10 AM stays 10 AM through DST flips. Keep tzdata fresh.
- **Q: Can you scale free-busy to 100 attendees?** A: Fan out in parallel; each is a single-shard read. Bitset-AND across attendees per day for O(N) merge. Cap meeting picker at ~50 attendees for sane UX.
- **Q: What if two people book the same slot?** A: Picker is eventually consistent; the actual write re-checks busy intervals and returns 409 + alternates.
- **Q: How do you sync to iPhone?** A: Implement CalDAV — PROPFIND for discovery, REPORT for time-range queries, sync tokens (RFC 6578) for incremental updates.
- **Q: A recurring rule has no UNTIL/COUNT, what now?** A: We cap at 1000 occurrences per request and reject century-spanning rules at write time.

KEY

Memorize These Numbers

500M users · ~1B active events · 500K reads/sec · 10K writes/sec · ~5 TB hot · ~150 GB Redis · 30-day reminder horizon · 4 ACL roles.